Sequential Task:

Multithreading 1

Object-Oriented Programming and GUI

05 May 2026

1. Declare a **LetterPrinter** class that implements the **Runnable** interface and performs cyclic character printing in a separate thread with synchronization control.

(a) Add a private field **letter** of type **cha**r that stores the character to be printed.

(b) Add a private field **running** of type boolean that controls the loop.

- Initialize the **running** field to **true**.

(c) Implement the **LetterPrinter(char letter)** constructor.

i. Accept a **letter** parameter.

ii. Assign its value to the **letter** field.

(d) Implement the **stopRunning()** method as **synchronized**.

i. Set the **running** field to **false**.

ii. Ensure safe modification of shared state between threads.

(e) Implement the private **isRunning()** method as **synchronized**.

i. Return the current value of the **running** field.

ii. Ensure safe reading of state in a multithreaded environment.

(f) Implement the **run()** method to define the logic executed in the thread.

i. Create a while loop that runs until the **isRunning()** method returns **true**.

ii. In each iteration, print the character stored in the **letter** field using **System.out.print(...)**.

iii. Pause the thread for one second.

- Call the **Thread.sleep(1000)** method.

iv. Handle the **InterruptedException** exception.

- If the thread is interrupted, terminate the loop using the **break** statement.

2. Declare a class **Letters** that implements the **Iterable<Thread>** interface and manages a set of threads and associated **LetterPrinter** objects.

(a) Add a private threads field of type **List<Thread>** that stores the created threads.

(b) Add aprivate printers field of type **List<LetterPrinter>**, which holds objects responsible for the character printing logic.

(c) Implement the **Letters(String letters)** constructor, whose task is to initialize structures and create threads.

i. Initialize the threads and printers lists as new **ArrayList** instances.

ii. Iterate through all the characters in the string **letters**.

• Retrieve a single character using **letters.charAt(i)**.

• Create a **LetterPrinter** object by passing the retrieved character.

• Create a new **Thread** thread by passing the **LetterPrinter** object and the thread name in the format **"Thread " + letter**.

• Add the created **LetterPrinter** object to the printers list.

•Add the created thread to the threads list.

(d) Implement the **start()** method, whose task is to start all threads.

i. Iterate through the threads list.

ii. Call the **start()** method on each thread.

(e) Implement the **stop()** method, which stops all threads.

i. Iterate through the list of printers.

ii. Call the **stopRunning()** method on each **LetterPrinter** object.

iii. Iterate through the list of threads.

iv. Call the **interrupt()** method on each thread to interrupt its execution.

(f) Implement the **iterator()** method derived from the **Iterable<Thread>** interface.

i. Return an iterator obtained from the list of threads using **threads.iterator()**.

3. Milestone 1: Present the results to the instructor.